

# **From Bits to Bootstrapping: A Bilingual, Hands-On Toolkit for Teaching nsF5 Steganography and Its Detection**

Fengjie Lin\*

13 September 2026

## **Summary**

`nsf5stego` is an open-source teaching and research toolkit for image steganography and steganalysis. It provides a complete, readable implementation of the nsF5 algorithm — Hamming syndrome matrix coding combined with wet paper coding (Westfeld, 2001; Fridrich et al., 2005) — together with SHA-256 content-keyed embedding, blind steganalysis by chi-square and RS analysis (Westfeld and Pfitzmann, 1999; Fridrich et al., 2001), and supervised detection with gradient-boosted classifiers (Ke et al., 2017). Around that core it ships a bilingual handbook (Chinese and English, 66 and 72 pages), ten runnable notebooks, an interactive browser laboratory, and a desktop GUI.

The design premise is that the three ideas which actually make modern steganography work — syndrome coding, the wet paper model, and the statistical footprint that embedding leaves behind — are learned by building them and then attacking what you built, not by reading about them. Every algorithm in the toolkit is therefore exposed as an object the learner can inspect and modify at intermediate steps, rather than as a library call that returns a stego image.

## **Statement of need**

Steganography teaching material tends to sit at one of two extremes. Textbooks treat it as mathematics: they derive the syndrome equation and the wet paper model rigorously, but leave the reader without a running pipeline (Fridrich, 2009; Provos and Honeyman, 2003). Course modules that do provide code tend to stop at LSB substitution, which is the one technique that the field moved past two decades ago precisely because it is trivially detectable. The result is that a student can often explain the Hamming bound while being unable to say what wet paper coding buys, why F5 shrinks the message, or what an embedding efficiency of 4 bits per change actually costs.

There is a second, subtler gap. Detection performance is overwhelmingly reported on curated benchmark corpora built for the purpose (Bas et al., 2011), where source images are partitioned and cover/stego pairs are carefully constructed (Pevný et al., 2010; Fridrich and Kodovský, 2012). Learners who train on their own photographs, or who split cover/stego pairs from the same source image across training and test sets, obtain numbers that look excellent and mean nothing. `nsf5stego` is built so that this mistake is difficult to make and easy to see: the dataset builder emits a `photo_id` for every sample, and every evaluation path groups by it by default. An audit of the project’s own experiments found a published comparison in which a 53-dimensional result had in fact been computed from 11 features — the exact class of error the toolkit now guards against with hard assertions rather than silent fallbacks.

## **Target audience and instructional context**

The material targets advanced undergraduates and beginning graduate students in security, signal processing, or applied machine learning. It assumes introductory Python and no prior exposure to information hiding; the handbook opens with pixels and binary representation before introducing anything steganographic. It has been designed as a one-semester elective or as the implementation backbone of a capstone

---

\*Qilu University of Technology. ORCID: 0000-0000-0000-0000 (placeholder — to be filled before submission).

project, with a twelve-week quick-start path and a six-to-twelve-month deep track for students continuing into research.

## What the toolkit contains

The package is organised so that theory, implementation, and verification stay adjacent:

- **A reference implementation** of nsF5 with binary Hamming codes [ $n = 2^p - 1, k, 3$ ], magnitude-decrement modification, and a wet paper solver that resolves dry positions through progressively deeper GF(2) searches. Optional C++ accelerators are compiled from source on Windows, macOS, and Linux; when absent, equivalent pure-Python paths run automatically, so notebooks and containers work with no build step.
- **Attacks on what you just built.** Chi-square and RS analysis run live against the learner’s own embedded image, and the browser laboratory draws the mask of changed pixels next to the detector verdict, so the connection between “these LSBs moved” and “this detector fired” is visible rather than asserted.
- **An interactive browser laboratory** covering bit-plane extraction and weighted restacking, an embed/decode round trip with live detection, a walkthrough of syndrome coding, a wet-paper dry-position solver, a side-by-side comparison of naive LSB against the project’s real nsF5, an ML decision-threshold playground, and a payload scan computed from the project’s own statistics.
- **Ten notebooks** that execute as Colab or Jupyter sessions, each mapped to a handbook chapter (Pérez and Granger, 2007).
- **A narrated video series** in Chinese (eleven chapters, about 36 minutes total) for students who encounter the written material before the mathematics it depends on.
- **A dataset builder** that produces cover/stego corpora with per-source-image identifiers, and a downloader for the BOSSbase 1.01 benchmark (Bas et al., 2011).

## Learning outcomes

By working through the material, a student should be able to: implement and invert Hamming syndrome matrix coding, and compute its embedding efficiency; explain why F5 shrinks the message and how nsF5 removes that shrinkage at the cost of a denser search; derive and solve the wet paper system for a given set of dry positions; implement chi-square and RS detectors and interpret their disagreement; build a supervised steganalysis classifier with grouped cross-validation and report it with confidence intervals; and — the outcome the project most cares about — recognise when a reported detection result is an artefact of evaluation design rather than a property of the detector.

This last point is taught through a worked negative example. A CNN reimplement in the repository attains a chance-level AUC on both validation *and training* data, which the toolkit’s reporting makes immediately legible as non-convergence rather than as evidence about convolutional networks, and which would otherwise have been publishable as a misleading comparison.

## Relationship to existing resources

General introductions (Provos and Honeyman, 2003) and the standard monograph (Fridrich, 2009) supply the theory but not a pipeline. Attack implementations exist primarily as research code accompanying papers (Pevný et al., 2010; Fridrich and Kodovský, 2012), which is optimised for throughput rather than for inspection. Deep-learning steganalysis is well served by reference architectures (Xu et al., 2016; Ye et al., 2017), but those presuppose the classical material this toolkit supplies. `nsf5stego` differs by covering the whole arc — pixel representation, embedding, keying, detection, classifier evaluation — in one inspectable codebase, and by treating the evaluation protocol itself as part of the curriculum.

## Verification and current use

The software is Apache-2.0 licensed and archived with a DOI. Twenty-four automated tests run under continuous integration on Ubuntu, macOS, and Windows, alongside a container build and a browser regression suite that exercises the interactive laboratory. The C++ and Python embedding paths are verified to interoperate, and the project documents where they legitimately differ rather than asserting equivalence.

The material has not yet been adopted in a formal course; the toolkit’s verification evidence at this stage concerns the software’s correctness, not its pedagogical effectiveness. The author welcomes collaboration with instructors willing to trial it, and would particularly value reports of the misconceptions students actually hold, which is the evidence the project currently lacks.

## Acknowledgements

The toolkit builds on the published steganography and steganalysis literature cited above, and on the NumPy and Matplotlib scientific Python stack (Harris et al., 2020). BOSSbase 1.01 is used as the reference benchmark (Bas et al., 2011).

## References

- Patrick Bas, Tomáš Filler, and Tomáš Pevný. Break our steganographic system: The ins and outs of organizing BOSS. In *Information Hiding (IH 2011)*, volume 6958 of *LNCS*, pages 59–70, 2011. doi: 10.1007/978-3-642-24178-9\_5.
- Jessica Fridrich. *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press, 2009. doi: 10.1017/CBO9781139192903.
- Jessica Fridrich and Jan Kodovský. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3):868–882, 2012. doi: 10.1109/TIFS.2012.2190402.
- Jessica Fridrich, Miroslav Goljan, and Rui Du. Reliable detection of LSB steganography in color and grayscale images. *IEEE Multimedia*, 8(4):22–28, 2001. doi: 10.1109/93.959097.
- Jessica Fridrich, Miroslav Goljan, Petr Lisonek, and David Soukal. Writing on wet paper. *IEEE Transactions on Signal Processing*, 53(10):3923–3935, 2005. doi: 10.1109/TSP.2005.855393.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 3146–3154, 2017.
- Fernando Pérez and Brian E. Granger. IPython: A system for interactive scientific computing. *Computing in Science & Engineering*, 9(3):21–29, 2007. doi: 10.1109/MCSE.2007.53.
- Tomáš Pevný, Patrick Bas, and Jessica Fridrich. Steganalysis by subtractive pixel adjacency matrix. *IEEE Transactions on Information Forensics and Security*, 5(2):215–224, 2010. doi: 10.1109/TIFS.2010.2045842.
- Niels Provos and Peter Honeyman. Hide and seek: An introduction to steganography. *IEEE Security & Privacy*, 1(3):32–44, 2003. doi: 10.1109/MSECP.2003.1203220.
- Andreas Westfeld. F5—a steganographic algorithm: High capacity despite better steganalysis. In *Information Hiding (IH 2001)*, volume 2137 of *LNCS*, pages 289–302, 2001. doi: 10.1007/3-540-45496-9\_21.
- Andreas Westfeld and Andreas Pfitzmann. Attacks on steganographic systems. In *Information Hiding (IH 1999)*, volume 1768 of *LNCS*, pages 61–76, 1999. doi: 10.1007/10719724\_5.
- Guanshuo Xu, Han-Zhou Wu, and Yun-Qing Shi. Structural design of convolutional neural networks for steganalysis. *IEEE Signal Processing Letters*, 23(5):708–712, 2016. doi: 10.1109/LSP.2016.2548421.
- Jian Ye, Jiangqun Ni, and Yang Yi. Deep learning hierarchical representations for image steganalysis. *IEEE Transactions on Information Forensics and Security*, 12(11):2545–2557, 2017. doi: 10.1109/TIFS.2017.2710950.